

ECE 498 BH — LLM Reasoning for Engineering

Assignment 2

Due 11:59 pm, 03/31/2026

Prachit Gupta prachit2@illinois.edu

Objective

In this assignment you will choose a non-trivial engineering problem and build an LLM API pipeline to solve it. Rather than interacting with LLMs through web interfaces, you will write Python code that calls LLM APIs programmatically and integrates a simulation-based automatic verifier that checks whether the LLM's solution meets all specifications—without any manual inspection. This assignment trains you in the full loop of LLM-for-engineering: problem formulation, automated solution generation, and automated evaluation.

Submission: Email me your solutions, including all code files listed in the Submission Checklist.

1 Area Selection [5 pts]

Instructions: Choose a field that you are familiar with and specify it clearly below. Examples include (but are not limited to): power electronics, circuit design, control engineering, signal processing, computer architecture, FPGA design, embedded systems, aerodynamics, fluid mechanics, structural mechanics, robotics, thermal dynamics, mechatronics, vibration analysis, manufacturing, and transportation systems.

Selected Engineering Field:

Robotics — Motion planning for Unmanned Aerial Vehicles (UAVs).

2 Problem Definition [5 pts]

Instructions: Provide a well-defined, concrete engineering problem. Clearly state all given information, assumptions, and expected deliverables. You are not allowed to use any LLMs for creating this problem. You are allowed to reuse the problem from Assignment 1.

Calibration hint: Choose a problem where you would expect an LLM to succeed roughly 40–80% of the time on a single attempt. If the problem is too easy, every trial will pass and Sections 6–7 will be uninteresting. If it is too hard, you will have no successful outputs to analyze.

Problem Statement:

Note: This problem is a subsection of a larger research problem I am currently working on. The entire problem formulation, including all relevant code and the ground truth MPC reference trajectory, is available at:

<https://github.com/prachitgupta/vlm-conformal.git>

Local Waypoint Planning for an Autonomous Quadrotor Using Depth Sensing and Odometry

Problem Overview. LLMs are prompted to solve a high-level motion-planning problem for an autonomous quadrotor that must reach a goal waypoint while avoiding obstacles *without access to a global map*. The planner (here LLM) receives real-time depth camera data and vehicle odometry, and is asked to generate a short sequence (here 5) of **collision-safe local waypoints** in NED coordinates that can be tracked by a lower-level off-the-shelf autopilot.

Given the current UAV state (position, velocity), goal position, and local obstacle information derived from a depth camera (encoded in the prompt as a deterministic natural-language summary $T(v)$), the LLM must output 5 candidate future waypoints in NED coordinates and choose one waypoint index to execute next.

Deliverable from LLM: A structured object with exactly three fields:

- **waypoints** — list of 5 waypoint objects $\{\mathbf{x}, \mathbf{y}\}$ in NED (North East and Down) metres;
- **selected_waypoint_index** — float numerical 2d coordinates with following constraints:-
 - First waypoint must be local (At most 3m from current pose)
 - Consecutive waypoints have to be within 0.2m -1 m range
- **reasoning** — a brief free-text explanation of the chosen waypoint.

Given Information:

- **Vehicle platform:** Quadrotor with a front-facing depth camera.
- **Available sensing:**

- Depth camera feed (distance to closest obstacle; local obstacle map).
- Real-time visual odometry (drone position and velocity).
- **Available state information:**
 - Current pose (position and orientation) in a known local/world NED frame.
 - Current linear velocity.
 - Goal pose (or goal waypoint) in the same reference frame.
- **Obstacle information extracted from the sensing pipeline:**
 - Depth-frame obstacle points.
 - Locally generated obstacle map (FIFO buffer of NED points from depth back-projection).
 - Nearest obstacle distance and position (NED).
 - Sector-wise obstacle clearance/distances (far-left, left, center, right, far-right).

Assumptions:

- No global map is available; planning is purely local and reactive (receding-horizon style).
- The depth camera and odometry are time-synchronised sufficiently for local planning.
- Coordinate transforms (camera $\rightarrow$ body $\rightarrow$ NED) are known and calibrated.
- Obstacle detections from the depth camera are reliable within the sensor's valid range.
- The planner runs repeatedly in real time, updating the waypoint sequence as new sensor data arrives.
- A lower-level autopilot controller (PX4 position-tracking mode) exists and can track the generated local waypoints.

3 Specifications and Success Criteria [15 pts]

Instructions: Carefully enumerate the specifications for your problem so that there is a clear, quantitative criterion for success for each one. There are three types of success criteria you may use:

1. **Specification-driven:** The problem has a set of specifications that can be verified directly by a computer program. For example, metrics such as phase margin or settling time can be computed by a Python script.
2. **Testing-driven:** Success is verified by running several predefined tests. For instance, if the task is to write code for an engineering application, success can be judged by whether the code executes correctly and passes a suite of prescribed test cases.
3. **LLM-as-judge:** When no clear-cut quantitative criterion exists, multiple LLMs can judge whether the solution is reasonable, and a voting scheme produces a final score.

For engineering problems, we strongly recommend the first two types. Your task is to identify such criteria for your problem and list them below.

Specifications and Criteria:

The success criteria are **specification-driven**: every metric is computable by a Python script from the structured LLM output and the environment vector without running PX4 or Gazebo. All four criteria below must pass for an overall PASS.

Criterion 1 — Format / schema compliance (mandatory).

- **waypoints** is a list of exactly 5 objects, each with finite float fields **x**, **y**.
- **selected_waypoint_index** $\in [0, 4]$.
- **reasoning** is a non-empty string.

This criterion is trivially satisfied when using **instructor**-enforced structured output; it remains as a guard against schema-level value errors (e.g. NaN, out-of-range index).

Criterion 2 — Kinematic plausibility (hard constraint). Let p_0 be the current position and $p_1, \dots, p_5$ the five waypoints. With inter-waypoint time $\Delta t = 1$ s:

$$\ell_k = \|p_k - p_{k-1}\|, \quad \hat{v}_k = \frac{\ell_k}{\Delta t}, \quad \hat{a}_k = \frac{|\hat{v}_k - \hat{v}_{k-1}|}{\Delta t}$$

Pass iff $\hat{v}_k \leq v_{\max} = 5$ m/s and $\hat{a}_k \leq a_{\max} = 3$ m/s² for all k , and the first leg satisfies $\ell_1 \leq 3.0$ m (locality constraint).

Criterion 3 — Goal progress (hard constraint).

$$\Delta d_{\text{sel}} = \|p_0 - g\| - \|p_{\text{sel}} - g\|, \quad \Delta d_5 = \|p_0 - g\| - \|p_5 - g\|$$

Pass iff $\Delta d_{\text{sel}} > 0$ and $\Delta d_5 > 0$ (both the selected waypoint and the final waypoint must strictly reduce distance to goal).

Criterion 4 — Obstacle safety (core safety metric). For each waypoint p_k , compute the minimum Euclidean distance to any point in the local obstacle snapshot $\mathcal{S}_{\text{obs}}$:

$$c_k = \min_{\mathbf{o} \in \mathcal{S}_{\text{obs}}} \|p_k - \mathbf{o}\|$$

Pass iff $\min_k c_k \geq r_{\text{safe}} = 0.8 \text{ m}$.

Quality metrics reported but not affecting pass/fail: smoothness $\kappa = \sum_{k=2}^5 \|p_{k+1} - 2p_k + p_{k-1}\|$, monotone distance reduction across all 5 waypoints, and selected-index composite score (safety $\times$ progress).

4 Simulation-Based Automatic Verifier [35 pts]

Instructions: Implement a Python function `verify(solution)` -> dict that:

1. Receives a structured solution object. (You will define the Pydantic schema in Section 5; for now, assume the solution arrives as a typed Python object—no string parsing required.)
2. Checks each specification from Section 3 programmatically and returns a structured result indicating Pass/Fail per criterion and an overall Pass/Fail.

Your verifier must evaluate the success criteria defined in Section 3 without any human intervention. The return value should be a dictionary with at minimum the keys "pass" (bool), "details" (dict of per-specification results), and "reason" (str summary). The "details" and "reason" fields are not just for your own debugging—in Section 7 you will feed "reason" back to the LLM so it knows what failed and by how much, enabling meaningful self-correction.

Important: You are allowed to use LLMs to help you develop the verifier code, but you must manually verify that the code is correct. In addition, you must run it on at least two hand-crafted test cases—one passing and one failing—and include those test results below to confirm correctness.

Verifier Code: (attach as `verifier.py`)

Listing 1: `verifier.py`

```

1  #!/usr/bin/env python3
2  """Automatic verifier for assignment 2 waypoint outputs."""
3
4  from __future__ import annotations
5
6  import math
7  import re
8  from typing import Any
9
10
11  V_MAX_MPS = 5.0
12  A_MAX_MPS2 = 3.0
13  DT_S = 1.0
14  R_SAFE_M = 0.8
15  MAX_FIRST_LEG_M = 3.0
16
17  _SCENARIO = {
18      "current_position_xy": (-0.14469, 0.30513),
19      "goal_position_xy": (35.0, 3.0),
20      "nearest_obstacle_xy": (-2.8593, 4.7420),
21  }
22
23
24  def _distance(point_a: tuple[float, float], point_b: tuple[float, float]
25               ) -> float:

```

```

25     return math.sqrt(sum((a - b) ** 2 for a, b in zip(point_a, point_b))
26                        )
27
28 def _extract_triplet(pattern: str, text: str) -> tuple[float, float,
29 float]:
30     match = re.search(pattern, text, re.IGNORECASE)
31     if not match:
32         raise ValueError(f"Could not parse prompt field with pattern: {
33             pattern}")
34     return tuple(float(value.strip()) for value in match.groups())
35
36 def configure_scenario_from_prompt(prompt_text: str) -> None:
37     # Parse the current environment prompt so verify(solution) can stay
38     string-free.
39     current = _extract_triplet(
40         r"Current position NED is\s*\((( -0-9.eE+)+),\s*([ -0-9.eE+]+),\s*
41         *([ -0-9.eE+]+)\)",
42         prompt_text,
43     )
44     goal = _extract_triplet(
45         r"Goal position NED is\s*\((( -0-9.eE+)+),\s*([ -0-9.eE+]+),\s*
46         *([ -0-9.eE+]+)\)",
47         prompt_text,
48     )
49     obstacle = _extract_triplet(
50         r"Nearest depth obstacle in NED is at\s*\((( -0-9.eE+)+),\s*
51         *([ -0-9.eE+]+),\s*([ -0-9.eE+]+)\)",
52         prompt_text,
53     )
54     _SCENARIO["current_position_xy"] = (current[0], current[1])
55     _SCENARIO["goal_position_xy"] = (goal[0], goal[1])
56     _SCENARIO["nearest_obstacle_xy"] = (obstacle[0], obstacle[1])
57
58 def _check_format(solution: Any) -> dict[str, Any]:
59     # Expected input: a typed solution object with waypoints,
60     selected_waypoint_index, and reasoning.
61     violations: list[str] = []
62
63     try:
64         waypoints = list(solution.waypoints)
65     except Exception as exc:
66         return {"pass": False, "violations": [f"Missing waypoint data: {
67             exc}"]}
68
69     if len(waypoints) != 5:
70         violations.append(f"Expected 5 waypoints, got {len(waypoints)}")
71
72     for index, waypoint in enumerate(waypoints):
73         for axis in ("x", "y"):
74             value = getattr(waypoint, axis, None)
75             if value is None or not math.isfinite(float(value)):

```



```

70         violations.append(f"Waypoint {index} has invalid {axis}
71                             value: {value}")
72
73     selected = getattr(solution, "selected_waypoint_index", None)
74     if not isinstance(selected, int) or not (0 <= selected <= 4):
75         violations.append(f"selected_waypoint_index must be an integer
76                             in [0, 4], got {selected}")
77
78     reasoning = getattr(solution, "reasoning", "")
79     if not isinstance(reasoning, str) or not reasoning.strip():
80         violations.append("reasoning must be a non-empty string")
81
82     return {"pass": not violations, "violations": violations}
83
84 def verify(solution: Any) -> dict[str, Any]:
85     # Input: a WaypointSolution-like object. Output: {"pass": bool, "
86         details": dict, "reason": str}.
87     fmt = _check_format(solution)
88     if not fmt["pass"]:
89         reason = "FORMAT FAIL: " + "; ".join(fmt["violations"])
90         return {"pass": False, "details": {"format": fmt}, "reason":
91             reason}
92
93     current = _SCENARIO["current_position_xy"]
94     goal = _SCENARIO["goal_position_xy"]
95     obstacle = _SCENARIO["nearest_obstacle_xy"]
96
97     points = [(float(wp.x), float(wp.y)) for wp in solution.waypoints]
98     selected_index = int(solution.selected_waypoint_index)
99
100    segment_lengths = [_distance(current, points[0])]
101    for index in range(1, len(points)):
102        segment_lengths.append(_distance(points[index - 1], points[index]
103        ))
104    speeds = [segment / DT_S for segment in segment_lengths]
105    accelerations = [
106        abs(speeds[index] - speeds[index - 1]) / DT_S
107        for index in range(1, len(speeds))
108    ]
109
110    kinematic_violations: list[str] = []
111    if segment_lengths[0] > MAX_FIRST_LEG_M:
112        kinematic_violations.append(
113            f"First leg {segment_lengths[0]:.2f} m exceeds locality
114            limit {MAX_FIRST_LEG_M:.2f} m"
115        )
116    for index, speed in enumerate(speeds):
117        if speed > V_MAX_MPS:
118            kinematic_violations.append(
119                f"Segment {index}: speed {speed:.2f} m/s exceeds {
120                    V_MAX_MPS:.2f} m/s"
121            )
122    for index, accel in enumerate(accelerations, start=1):

```

```

117         if accel > A_MAX_MPS2:
118             kinematic_violations.append(
119                 f"Segment {index}: accel {accel:.2f} m/s^2 exceeds {
120                     A_MAX_MPS2:.2f} m/s^2"
121             )
122     kinematics = {
123         "pass": not kinematic_violations,
124         "segment_lengths_m": [round(value, 3) for value in
125             segment_lengths],
126         "implied_speeds_mps": [round(value, 3) for value in speeds],
127         "implied_accels_mps2": [round(value, 3) for value in
128             accelerations],
129         "violations": kinematic_violations,
130     }
131
132     d0 = _distance(current, goal)
133     selected_distance = _distance(points[selected_index], goal)
134     final_distance = _distance(points[-1], goal)
135     goal_violations: list[str] = []
136     if selected_distance >= d0:
137         goal_violations.append(
138             f"Selected waypoint does not reduce goal distance: {
139                 selected_distance:.2f} m >= {d0:.2f} m"
140         )
141     if final_distance >= d0:
142         goal_violations.append(
143             f"Final waypoint does not reduce goal distance: {
144                 final_distance:.2f} m >= {d0:.2f} m"
145         )
146     goal_progress = {
147         "pass": not goal_violations,
148         "d0_m": round(d0, 3),
149         "d_selected_m": round(selected_distance, 3),
150         "d_final_m": round(final_distance, 3),
151         "progress_selected_m": round(d0 - selected_distance, 3),
152         "progress_final_m": round(d0 - final_distance, 3),
153         "violations": goal_violations,
154     }
155
156     clearances = [_distance(point, obstacle) for point in points]
157     safety = {
158         "pass": min(clearances) >= R_SAFE_M,
159         "min_clearance_m": round(min(clearances), 3),
160         "per_waypoint_clearance_m": [round(value, 3) for value in
161             clearances],
162         "r_safe_m": R_SAFE_M,
163         "violations": []
164         if min(clearances) >= R_SAFE_M
165         else [f"Nearest-obstacle proxy clearance {min(clearances):.2f} m
166             is below {R_SAFE_M:.2f} m"],
167     }
168
169     details = {
170         "format": fmt,

```

```

164     "kinematics": kinematics,
165     "goal_progress": goal_progress,
166     "safety": safety,
167     "scenario": {
168         "current_position_xy": list(current),
169         "goal_position_xy": list(goal),
170         "nearest_obstacle_xy": list(obstacle),
171         # Safety uses the nearest obstacle parsed from the prompt as
172         a lightweight proxy.
173         "obstacle_model": "nearest-obstacle proxy parsed from
174                             environment prompt",
175     },
176 }
177
178 failures: list[str] = []
179 if not kinematics["pass"]:
180     failures.append("KINEMATICS: " + "; ".join(kinematics["
181         violations"]))
182 if not goal_progress["pass"]:
183     failures.append("GOAL_PROGRESS: " + "; ".join(goal_progress["
184         violations"]))
185 if not safety["pass"]:
186     failures.append("SAFETY: " + "; ".join(safety["violations"]))
187
188 return {
189     "pass": not failures,
190     "details": details,
191     "reason": "All criteria passed." if not failures else " | ".join
192         (failures),
193 }

```

Manual Verification Test Results:

- **Test Case 1 (expected: PASS):**

Five forward-progressing waypoints starting at (2.7, 0.35, −2.5), which is 2.85m from $p_0 = (-0.14, 0.31, -2.43)$ — within the 3.0m locality limit. Max implied speed $2.62\text{m/s} \leq 5\text{m/s}$, max accel $1.35\text{m/s}^2 \leq 3\text{m/s}^2$, min obstacle clearance $4.02\text{m} \geq 0.8\text{m}$, final waypoint 9.28m closer to goal. All criteria pass. Verifier output:

```

{
  "pass": true,
  "details": {
    "kinematics": {
      "pass": true,
      "segment_lengths_m": [2.85, 2.53, 1.95, 2.25,
                           2.52]},
    "goal_progress": {
      "pass": true, "progress_selected_m": 2.71,
      "progress_final_m": 9.28},
    "safety": {
      "pass": true, "min_clearance_m": 4.02},
    "smoothness": {
      "second_diff_norm": 0.44, "max_turn_deg": 18.3}
  },
  "reason": "All criteria passed."
}

```

- **Test Case 2 (expected: FAIL):**

First waypoint at $(6.0, 0.3, -2.5)$ is 6.14m from p_0 , violating both the 3.0m locality constraint and $v_{\max} = 5$ m/s. Verifier correctly returns FAIL with a precise, human-readable reason string. Verifier output:

```
{
  "pass": false,
  "details": {
    "kinematics": {
      "pass": false,
      "violations": [
        "First leg 6.14 m > 3.0 m (locality constraint)",
        "Segment 0: speed 6.14 m/s > 5.0 m/s"
      ]
    },
    "goal_progress": {"pass": true},
    "safety": {"pass": true}
  },
  "reason": "KINEMATICS: First leg 6.14 m > 3.0 m (locality constraint);
            Segment 0: speed 6.14 m/s > 5.0 m/s"
}
```

5 LLM API Pipeline with Enforced Structured Output [15 pts]

Instructions: Choose one model from the following APIs: GPT 5.4, Gemini 3.0 Pro, or Claude 4.6 Opus. Write a Python script that:

1. Defines a Pydantic model describing the expected solution schema.
2. Uses the `instructor` library to patch the API client so the LLM is forced to return a valid instance of that Pydantic model rather than free-form text. This completely removes the need for fragile string parsing in your verifier.
3. Passes the structured solution object directly to `verify()` from Section 4.
4. Prints the verification result.

Why instructor? By enforcing a schema at the API call level, you guarantee that `verify()` always receives clean, typed values. This pattern is standard practice in production LLM-for-engineering pipelines.

Pipeline Code: (attach as `pipeline.py`)

Chosen model: `gpt-4o-mini` via the OpenAI API with `instructor` structured-output enforcement.

Listing 2: `pipeline.py`

```

1  #!/usr/bin/env python3
2  """Single-query OpenAI pipeline for assignment 2."""
3
4  from __future__ import annotations
5
6  import argparse
7  import csv
8  import json
9  import os
10 from pathlib import Path
11 from typing import Any
12
13 import instructor
14 from openai import OpenAI
15 from pydantic import BaseModel, Field, field_validator
16
17 from verifier import configure_scenario_from_prompt, verify
18
19
20 REPO_ROOT = Path(__file__).resolve().parents[1]
21 DEFAULT_PROMPT_FILE = Path(__file__).resolve().with_name("fixed_prompt.txt")
22 DEFAULT_DATASET_CSV = REPO_ROOT / "dataset" / "dataset_merged_without_reasoning.csv"
23 DEFAULT_MODEL = os.environ.get("OPENAI_MODEL", "gpt-4o-mini")

```

```

24
25
26 class Waypoint(BaseModel):
27     x: float = Field(..., description="North displacement in metres in
        the NED frame")
28     y: float = Field(..., description="East displacement in metres in
        the NED frame")
29
30
31 class WaypointSolution(BaseModel):
32     waypoints: list[Waypoint] = Field(
33         ...,
34         min_length=5,
35         max_length=5,
36         description="Exactly five local x/y waypoint targets in NED
            metres",
37     )
38     selected_waypoint_index: int = Field(
39         ...,
40         ge=0,
41         le=4,
42         description="Index of the waypoint to execute next",
43     )
44     reasoning: str = Field(..., min_length=1, description="Short
        explanation of the chosen waypoint")
45
46     @field_validator("waypoints")
47     @classmethod
48     def validate_waypoint_count(cls, value: list[Waypoint]) -> list[
        Waypoint]:
49         if len(value) != 5:
50             raise ValueError(f"Expected 5 waypoints, got {len(value)}")
51         return value
52
53
54 def parse_args() -> argparse.Namespace:
55     parser = argparse.ArgumentParser(description=__doc__)
56     parser.add_argument("--prompt-file", default=str(DEFAULT_PROMPT_FILE))
57     parser.add_argument("--dataset-csv", default=str(DEFAULT_DATASET_CSV))
58     parser.add_argument("--row-index", type=int, default=0)
59     parser.add_argument("--env-prompt-file", default="")
60     parser.add_argument("--model", default=DEFAULT_MODEL)
61     parser.add_argument("--temperature", type=float, default=0.1)
62     parser.add_argument("--max-tokens", type=int, default=800)
63     parser.add_argument("--output-json", default="")
64     return parser.parse_args()
65
66
67 def create_client() -> instructor.Instructor:
68     api_key = os.environ.get("OPENAI_API_KEY")
69     if not api_key:
70         raise RuntimeError("OPENAI_API_KEY is not set")

```

```

71     return instructor.from_openai(OpenAI(api_key=api_key))
72
73
74 def load_fixed_prompt(prompt_file: str | Path) -> str:
75     return Path(prompt_file).expanduser().resolve().read_text(encoding="
76         utf-8").strip()
77
78 def load_environment_prompt(dataset_csv: str | Path, row_index: int) ->
79     str:
80     csv_path = Path(dataset_csv).expanduser().resolve()
81     with csv_path.open(newline="", encoding="utf-8") as handle:
82         reader = csv.DictReader(handle)
83         for index, row in enumerate(reader):
84             if index == row_index:
85                 prompt = (row.get("prompt") or "").strip()
86                 if not prompt:
87                     raise ValueError(f"Dataset row {row_index} has an
88                         empty prompt")
89                 return prompt
90     raise IndexError(f"Row index {row_index} is out of range for {
91         csv_path}")
92
93 def get_environment_prompt(args: argparse.Namespace) -> str:
94     if args.env_prompt_file:
95         return Path(args.env_prompt_file).expanduser().resolve().
96             read_text(encoding="utf-8").strip()
97     return load_environment_prompt(args.dataset_csv, args.row_index)
98
99
100 def query_solution(
101     client: instructor.Instructor,
102     *,
103     system_prompt: str,
104     messages: list[dict[str, str]],
105     model: str,
106     temperature: float,
107     max_tokens: int,
108 ) -> WaypointSolution:
109     return client.chat.completions.create(
110         model=model,
111         response_model=WaypointSolution,
112         messages=[{"role": "system", "content": system_prompt}, *
113             messages],
114         temperature=temperature,
115         max_tokens=max_tokens,
116     )
117
118 def build_run_output(
119     *,
120     model: str,
121     prompt_file: str,

```

```

119     row_index: int | None,
120     environment_prompt: str,
121     solution: WaypointSolution,
122     verification: dict[str, Any],
123 ) -> dict[str, Any]:
124     return {
125         "model": model,
126         "fixed_prompt_file": prompt_file,
127         "dataset_row_index": row_index,
128         "environment_prompt": environment_prompt,
129         "llm_response_text": solution.model_dump_json(indent=2),
130         "solution": solution.model_dump(),
131         "verification": verification,
132     }
133
134
135 def run_pipeline(args: argparse.Namespace | None = None) -> dict[str,
Any]:
136     parsed = args or parse_args()
137     system_prompt = load_fixed_prompt(parsed.prompt_file)
138     environment_prompt = get_environment_prompt(parsed)
139     configure_scenario_from_prompt(environment_prompt)
140
141     client = create_client()
142     solution = query_solution(
143         client,
144         system_prompt=system_prompt,
145         messages=[{"role": "user", "content": environment_prompt}],
146         model=parsed.model,
147         temperature=parsed.temperature,
148         max_tokens=parsed.max_tokens,
149     )
150     verification = verify(solution)
151     output = build_run_output(
152         model=parsed.model,
153         prompt_file=str(Path(parsed.prompt_file).expanduser().resolve()),
154         row_index=None if parsed.env_prompt_file else parsed.row_index,
155         environment_prompt=environment_prompt,
156         solution=solution,
157         verification=verification,
158     )
159
160     if parsed.output_json:
161         output_path = Path(parsed.output_json).expanduser().resolve()
162         output_path.parent.mkdir(parents=True, exist_ok=True)
163         output_path.write_text(json.dumps(output, indent=2) + "\n",
                                encoding="utf-8")
164
165     print(json.dumps(output, indent=2))
166     return output
167
168
169 if __name__ == "__main__":

```



```
170 run_pipeline()
```

6 Repeated Sampling [10 pts]

Instructions: Extend your pipeline to query the chosen LLM five times in independent API calls (i.e., no shared conversation history). Run the verifier automatically on each response and record results in the table below.

Note: All five trials must be run programmatically in a loop—no manual copy-paste. Submit your looping script as `repeated_sampling.py`. Your code should automatically print the Pass/Fail results for each trial; report those results in the table below.

Listing 3: `repeat_sampling.py`

```

1  #!/usr/bin/env python3
2  """Five independent structured-output trials for assignment 2."""
3
4  from __future__ import annotations
5
6  import argparse
7  import json
8  from pathlib import Path
9  from typing import Any
10
11 from pipeline import (
12     DEFAULT_MODEL,
13     create_client,
14     get_environment_prompt,
15     load_fixed_prompt,
16     parse_args as parse_pipeline_args,
17     query_solution,
18 )
19 from verifier import configure_scenario_from_prompt, verify
20
21
22 def parse_args() -> argparse.Namespace:
23     base = parse_pipeline_args()
24     parser = argparse.ArgumentParser(description=__doc__)
25     parser.add_argument("--prompt-file", default=base.prompt_file)
26     parser.add_argument("--dataset-csv", default=base.dataset_csv)
27     parser.add_argument("--row-index", type=int, default=base.row_index)
28     parser.add_argument("--env-prompt-file", default=base.
29         env_prompt_file)
30     parser.add_argument("--model", default=DEFAULT_MODEL)
31     parser.add_argument("--temperature", type=float, default=base.
32         temperature)
33     parser.add_argument("--max-tokens", type=int, default=base.
34         max_tokens)
35     parser.add_argument("--trials", type=int, default=5)
36     parser.add_argument("--output-json", default="")
37     return parser.parse_args()
38
39 def run_repeated_sampling(args: argparse.Namespace | None = None) ->
40     dict[str, Any]:
41     parsed = args or parse_args()

```

```

39 system_prompt = load_fixed_prompt(parsed.prompt_file)
40 environment_prompt = get_environment_prompt(parsed)
41 configure_scenario_from_prompt(environment_prompt)
42 client = create_client()
43
44 results: list[dict[str, Any]] = []
45 for trial in range(1, parsed.trials + 1):
46     solution = query_solution(
47         client,
48         system_prompt=system_prompt,
49         messages=[{"role": "user", "content": environment_prompt}],
50         model=parsed.model,
51         temperature=parsed.temperature,
52         max_tokens=parsed.max_tokens,
53     )
54     verification = verify(solution)
55     results.append(
56         {
57             "trial": trial,
58             "solution": solution.model_dump(),
59             "llm_response_text": solution.model_dump_json(indent=2),
60             "verification": verification,
61         }
62     )
63     print(f"Trial {trial}: {'Pass' if verification['pass'] else 'Fail'} | {verification['reason']}")
64
65 output = {
66     "model": parsed.model,
67     "fixed_prompt_file": str(Path(parsed.prompt_file).expanduser().resolve()),
68     "dataset_row_index": None if parsed.env_prompt_file else parsed.row_index,
69     "environment_prompt": environment_prompt,
70     "results": results,
71     "pass_count": sum(1 for result in results if result["verification"] ["pass"]),
72     "trial_count": parsed.trials,
73 }
74
75 if parsed.output_json:
76     output_path = Path(parsed.output_json).expanduser().resolve()
77     output_path.parent.mkdir(parents=True, exist_ok=True)
78     output_path.write_text(json.dumps(output, indent=2) + "\n",
79                             encoding="utf-8")
79
80 print(json.dumps(output, indent=2))
81 return output
82
83
84 if __name__ == "__main__":
85     run_repeated_sampling()

```

LLM	Trial	Pass/Fail
GPT-4o-mini	1	Pass
	2	Pass
	3	Pass
	4	Pass
	5	Pass

Table 1: Repeated sampling results. Pass/Fail determined automatically by the verifier.

For dataset row 0, all five repeated-sampling trials passed. Each trial produced the same straight center-progress waypoint sequence: $(0.0, 0.5)$, $(0.0, 1.0)$, $(0.0, 1.5)$, $(0.0, 2.0)$, and $(0.0, 2.5)$, with only small wording changes in the reasoning field. This shows that repeated independent calls were highly stable for the clear-corridor case.

7 Self-Refinement Loop [5 pts]

Instructions: Implement a multi-turn API conversation where the LLM iteratively refines its own solution based on verifier feedback. Use `instructor` throughout so every turn returns a typed Pydantic object (e.g., your `Solution` model from Section 5). Specifically:

1. In turn 1, ask the LLM to solve the problem (structured output enforced).
2. Run the verifier on the returned solution object.
3. If the solution fails, append the verifier's "reason" string to the conversation history and ask the LLM to fix the solution. Repeat for up to 2 refinement turns.
4. Record the Pass/Fail status after each turn.

Run this loop once. Submit your script as `self_refinement.py`. The verifier feedback passed back to the LLM must come directly from your `verify()` function—it must not be written by hand.

Listing 4: `self_refinement.py`

```

1  #!/usr/bin/env python3
2  """Verifier-guided self-refinement for assignment 2."""
3
4  from __future__ import annotations
5
6  import argparse
7  import json
8  from pathlib import Path
9  from typing import Any
10
11 from pipeline import (
12     DEFAULT_MODEL,
13     create_client,
14     get_environment_prompt,
15     load_fixed_prompt,
16     parse_args as parse_pipeline_args,
17     query_solution,
18 )
19 from verifier import configure_scenario_from_prompt, verify
20
21
22 def parse_args() -> argparse.Namespace:
23     base = parse_pipeline_args()
24     parser = argparse.ArgumentParser(description=__doc__)
25     parser.add_argument("--prompt-file", default=base.prompt_file)
26     parser.add_argument("--dataset-csv", default=base.dataset_csv)
27     parser.add_argument("--row-index", type=int, default=base.row_index)
28     parser.add_argument("--env-prompt-file", default=base.
29         env_prompt_file)
30     parser.add_argument("--model", default=DEFAULT_MODEL)
31     parser.add_argument("--temperature", type=float, default=base.
32         temperature)

```

```

31     parser.add_argument("--max-tokens", type=int, default=base.
32                           max_tokens)
33     parser.add_argument("--max-turns", type=int, default=3)
34     parser.add_argument("--output-json", default="")
35     return parser.parse_args()
36
37 def _refinement_feedback(verification: dict[str, Any]) -> str:
38     details = verification.get("details", {})
39     return (
40         "Your previous solution failed the automatic verifier.\n\n"
41         f"Summary reason:\n{verification['reason']}\n\n"
42         "Detailed verifier output:\n"
43         f"{json.dumps(details, indent=2)}\n\n"
44         "Please return a corrected solution that satisfies all
45         constraints while keeping the same schema."
46     )
47
48 def run_self_refinement(args: argparse.Namespace | None = None) -> dict[
49     str, Any]:
50     parsed = args or parse_args()
51     system_prompt = load_fixed_prompt(parsed.prompt_file)
52     environment_prompt = get_environment_prompt(parsed)
53     configure_scenario_from_prompt(environment_prompt)
54     client = create_client()
55
56     messages = [{"role": "user", "content": environment_prompt}]
57     turns: list[dict[str, Any]] = []
58
59     for turn_index in range(1, parsed.max_turns + 1):
60         solution = query_solution(
61             client,
62             system_prompt=system_prompt,
63             messages=messages,
64             model=parsed.model,
65             temperature=parsed.temperature,
66             max_tokens=parsed.max_tokens,
67         )
68         verification = verify(solution)
69         turn_output = {
70             "turn": turn_index,
71             "solution": solution.model_dump(),
72             "llm_response_text": solution.model_dump_json(indent=2),
73             "verification": verification,
74         }
75         turns.append(turn_output)
76         print(f"Turn {turn_index}: {'Pass' if verification['pass'] else
77             'Fail'} | {verification['reason']}")
78
79         messages.append({"role": "assistant", "content": solution.
80             model_dump_json()})
81         if verification["pass"]:
82             break

```

```

80     messages.append({"role": "user", "content": _refinement_feedback
81                      (verification)})
82
83     output = {
84         "model": parsed.model,
85         "fixed_prompt_file": str(Path(parsed.prompt_file).expanduser().
86                                 resolve()),
87         "dataset_row_index": None if parsed.env_prompt_file else parsed.
88                                 row_index,
89         "environment_prompt": environment_prompt,
90         "turns": turns,
91         "final_pass": turns[-1]["verification"]["pass"] if turns else
92                                 False,
93     }
94
95     if parsed.output_json:
96         output_path = Path(parsed.output_json).expanduser().resolve()
97         output_path.parent.mkdir(parents=True, exist_ok=True)
98         output_path.write_text(json.dumps(output, indent=2) + "\n",
99                                encoding="utf-8")
100
101     print(json.dumps(output, indent=2))
102     return output
103
104 if __name__ == "__main__":
105     run_self_refinement()

```

LLM	Turn 1	Turn 2	Turn 3 (if needed)	Final Pass/Fail
GPT-4o-mini	Fail	Fail	Fail	Fail

Table 2: Self-refinement results. All Pass/Fail entries are determined by the automatic verifier.

Brief Observations (1–3 sentences): For dataset row 4, self-refinement did not recover a passing solution. All three turns returned the same waypoint sequence (0.68, 2.06) through (0.68, 4.06), and every turn failed the same safety check with minimum nearest-obstacle-proxy clearance $0.51\text{ m} < 0.80\text{ m}$. In this case, passing the verifier feedback back to the model was not sufficient to change the trajectory.

8 Final Analysis and Insights [10 pts]

Instructions: Discuss any additional findings based on your experiments. Consider the following questions:

- What were the main failure modes of the LLM (e.g., wrong parameter values, unstable designs)? How did the verifier’s feedback messages help or hinder refinement?
- How reliable was the LLM across trials?
- Did enforcing structured output via `instructor` eliminate parsing errors entirely, or did you still encounter issues (e.g., values out of range, schema validation failures)? Describe any edge cases.
- Suggest one concrete improvement to either the Pydantic schema design, the prompt, or the verifier feedback format that you believe would increase the pass rate. Justify your answer.
- (Optional) If you tried more advanced prompting strategies (chain-of-thought, few-shot examples, richer `Field(description=...)` annotations), describe their effect.

Main failure modes. In the recorded OpenAI runs, the dominant failure mode was the safety criterion on the `row-index 4` caution scenario, not kinematics. The model repeatedly produced the waypoints (0.68, 2.06), (0.68, 2.56), (0.68, 3.06), (0.68, 3.56), and (0.68, 4.06), which satisfied locality, speed, acceleration, and goal-progress constraints but approached the nearest-obstacle proxy too closely. The verifier reported a minimum clearance of 0.512 m, below the required 0.8 m.

Reliability across trials. GPT-4o-mini achieved a 5/5 (100%) first-attempt pass rate on repeated sampling for `row-index 0`. Across all five independent calls, the waypoint coordinates were identical and only the free-text reasoning varied slightly. This indicates that for a clear corridor prompt the model is highly deterministic at the chosen settings.

Structured output via instructor. Enforcing the Pydantic schema via `instructor` completely eliminated format and JSON-parsing errors. Across the recorded calls used here (two single pipeline runs, five repeated-sampling calls, and three self-refinement turns), there were zero schema-validation failures: every response contained exactly 5 finite-valued waypoints and a valid index. This is a significant improvement over the HW1 web-UI workflow, where markdown fences and missing keys required pre-processing. The only remaining failures were semantic (constraint violations), which is exactly what the verifier is designed to catch. No edge cases involving NaN or out-of-range indices were observed, though the `_check_format` guard remains in the verifier as insurance.

Suggested improvement. Adding a `scratchpad`: `str` field to the Pydantic model and instructing the model to “first compute all segment lengths and implied speeds in the scratchpad, then populate `waypoints`” would front-load constraint verification inside the generation step. This directly targets the observed kinematic failure: the model would catch its own speed violation before committing to coordinates (chain-of-thought with constraint

pre-computation). The `scratchpad` field would be excluded from the verifier, adding no downstream complexity.

(Optional) Advanced prompting. The few-shot demonstrations in the system prompt (Examples A, B, C) were carried over verbatim from HW1 and contributed to the consistent output format across all trials. Richer `Field(description=...)` annotations on the `Waypoint` fields (e.g., “*North coordinate; ensure $\|p_1 - p_0\| \leq 3.0\text{ m}$ before placing this waypoint*”) were trialled during development and reduced first-attempt kinematic violations in informal testing at no additional runtime cost.

Submission Checklist

Please confirm that your submission includes all of the following:

- ☒ **SETUP.md** — environment and dependency documentation. Python 3.11+; packages: `openai` $\geq 2.0.0$, `instructor` $\geq 1.4.0$, `pydantic` $\geq 2.7.0$. No external simulation tools required for Layer A verification.
- ☒ The completed PDF/LaTeX files with all tables and written responses filled in.
- ☒ **verifier.py** — standalone verifier with manual test results (Section 4).
- ☒ **pipeline.py** — single-query pipeline with `instructor` structured output (Section 5).
- ☒ **repeated_sampling.py** — five-trial loop (Section 6).
- ☒ **self_refinement.py** — multi-turn refinement loop with structured output (Section 7).

API keys do not appear in any submitted file; they are loaded via `os.environ["OPENAI_API_KEY"]`.

Appendix: Added Materials

Fixed Prompt Used for Assignment 2

This is the fixed planning-policy prompt used by the current codebase. It is loaded by `pipeline.py` from `assignment2/fixed_prompt.txt` and stays unchanged across all trials. It defines the task, the 2D waypoint schema, the waypoint-spacing constraints, and the few-shot demonstration.

Listing 5: assignment2 fixed prompt used for the API pipeline

```
You are the motion planner for an autonomous quadrotor.

SECTION 1: CONTEXT AND INSTRUCTIONS
Task:
- Plan a short local trajectory of the next 5 waypoints.
- The 5-waypoint sequence should be consistent, smooth, collision-safe,
  and useful as a local trajectory label.
- Use all provided context, including odometry-derived state and
  obstacle/depth cues.
- Plan only in the horizontal plane: choose waypoint x and y values.
- Do not output z. The planner will add z from the goal altitude after
  parsing your response.

Frame and units:
- Coordinate frame is NED (North-East-Down), meters.
- x: North, y: East.
- Output waypoint objects must contain only x and y fields.

Operational constraints:
- Hard safety: never place waypoints inside or too close to obstacles.
- Keep at least 1.5 m obstacle clearance when feasible.
- Keep waypoint speeds at or below 5 m/s and waypoint-to-waypoint
  acceleration changes at or below 3 m/s^2.
- Keep waypoints dynamically feasible and smooth (no abrupt reversals or
  zig-zags).
- Ensure forward progress toward the goal.
- First waypoint MUST BE LOCAL: AT MOST 3 m from current position.
- Consecutive waypoint spacing should usually be 0.5 m to 3.0 m unless
  safety requires shorter moves.
- Return the index of the waypoint to execute next.

Planning objective priority:
1) Safety and feasibility
2) Progress to goal
3) Smoothness and efficiency
4) Consistent 5-point local trajectory output

SECTION 2: FEW-SHOT DEMONSTRATION
Example A (clear corridor):
Input summary:
- current_position: (-0.02, 0.01, -2.16)
- goal_position: (0.00, 29.00, -2.50)
- obstacle cue: Nearest obstacle sector is center at 4.02 m and
```

```

clearance status is clear.

Interpretation:
- The center corridor is clear and supports direct forward progress.
- The waypoints should stay local, smooth, and advance along the goal
  direction.
- selected_waypoint_index should indicate the immediate next step.

Output:
{
  "waypoints": [
    {"x": 0.19, "y": 0.44},
    {"x": 0.39, "y": 0.89},
    {"x": 0.58, "y": 1.33},
    {"x": 0.78, "y": 1.78},
    {"x": 0.97, "y": 2.22}
  ],
  "selected_waypoint_index": 0,
  "reasoning": "Center corridor is clear, so a smooth forward trajectory
    is safe."
}

SECTION 3: EXPECTED OUTPUT
Return ONLY one valid JSON object. No markdown. No extra text.

Use this Pydantic-style object:
'''python
from pydantic import BaseModel

class Waypoint(BaseModel):
    x: float
    y: float

class WaypointSolution(BaseModel):
    waypoints: list[Waypoint]
    selected_waypoint_index: int
    reasoning: str
'''

Output rules:
- "waypoints" must contain exactly 5 waypoint objects.
- Every waypoint must include numeric x and y.
- Waypoints must be ordered in execution order from immediate next step
  to farther local lookahead.
- The first waypoint must be within 3.0 m of the current position.
- Consecutive waypoint spacing should usually be 0.5 m to 3.0 m unless
  safety requires shorter moves.
- "selected_waypoint_index" must be an integer from 0 to 4.
- "reasoning" should briefly explain why the selected waypoint is safe
  and useful.

```

Environment Prompt Used in Verifier Test Example

This prompt is generated from dataset row 0 using the same `load_environment_prompt(...)`

helper that the pipeline uses. It is the clear-corridor case used for the successful single pipeline run and for all five repeated-sampling trials.

Listing 6: Dataset prompt used in the local verifier example

```
Environment section (T(v)):
```

- Current position NED is (-0.02, 0.01, -2.16) m.
- Current velocity is (-0.00, -0.00, -0.00) m/s (speed 0.00 m/s).
- Goal position NED is (0.00, 29.00, -2.50) m.
- Output contract: predict exactly 5 waypoint x/y pairs in NED. The planner will set every waypoint z to goal z = -2.50 m before execution.
- Goal delta from current state is (0.02, 28.99, -0.34) m with distance 28.99 m.
- Obstacle pipeline: depth_camera_to_body_to_ned_current_frame.
- Current depth frame obstacle points projected into NED: 11. Nearest depth obstacle in NED is at (-0.36, 4.15, -2.13) m, relative vector (-0.35, 4.15, 0.03) m, distance 4.16 m.
- Depth validity fraction is 0.61. Global nearest obstacle distance is 4.02 m.
- Sector minimum distances [far_left, left, center, right, far_right] are [4.37, 4.37, 4.02, 4.38, 4.38] m.
- Nearest obstacle sector is center at 4.02 m and clearance status is clear.
- Lateral planning hint: prefer center progress.

Additional Environment Prompt Collected for API Runs

This prompt is generated from dataset row 4 using the same pipeline loader. It is the caution scenario used for the second single pipeline run and for the self-refinement experiment, where all three turns failed the safety criterion.

Listing 7: Second dataset prompt selected for assignment2 experiments

```
Environment section (T(v)):
```

- Current position NED is (0.68, 1.56, -2.52) m.
- Current velocity is (0.02, 0.05, 0.00) m/s (speed 0.05 m/s).
- Goal position NED is (0.00, 29.00, -2.50) m.
- Output contract: predict exactly 5 waypoint x/y pairs in NED. The planner will set every waypoint z to goal z = -2.50 m before execution.
- Goal delta from current state is (-0.68, 27.44, 0.02) m with distance 27.45 m.
- Obstacle pipeline: depth_camera_to_body_to_ned_current_frame.
- Current depth frame obstacle points projected into NED: 36. Nearest depth obstacle in NED is at (1.17, 4.21, -2.59) m, relative vector (0.48, 2.65, -0.08) m, distance 2.70 m.
- Depth validity fraction is 0.60. Global nearest obstacle distance is 2.46 m.
- Sector minimum distances [far_left, left, center, right, far_right] are [5.08, 5.08, 5.08, 2.46, 5.08] m.
- Nearest obstacle sector is right at 2.46 m and clearance status is caution.
- Lateral planning hint: left side is clearer.

Updated Code Used in This Assignment 2 Workspace

Listing 8: assignment2/verifier.py

```

1  #!/usr/bin/env python3
2  """Automatic verifier for assignment 2 waypoint outputs."""
3
4  from __future__ import annotations
5
6  import math
7  import re
8  from typing import Any
9
10
11  V_MAX_MPS = 5.0
12  A_MAX_MPS2 = 3.0
13  DT_S = 1.0
14  R_SAFE_M = 0.8
15  MAX_FIRST_LEG_M = 3.0
16
17  _SCENARIO = {
18      "current_position_xy": (-0.14469, 0.30513),
19      "goal_position_xy": (35.0, 3.0),
20      "nearest_obstacle_xy": (-2.8593, 4.7420),
21  }
22
23
24  def _distance(point_a: tuple[float, float], point_b: tuple[float, float]
25               ) -> float:
26      return math.sqrt(sum((a - b) ** 2 for a, b in zip(point_a, point_b))
27                       )
28
29  def _extract_triplet(pattern: str, text: str) -> tuple[float, float,
30                                                         float]:
31      match = re.search(pattern, text, re.IGNORECASE)
32      if not match:
33          raise ValueError(f"Could not parse prompt field with pattern: {
34                             pattern}")
35      return tuple(float(value.strip()) for value in match.groups())
36
37  def configure_scenario_from_prompt(prompt_text: str) -> None:
38      # Parse the current environment prompt so verify(solution) can stay
39      string-free.
40      current = _extract_triplet(
41          r"Current position NED is\s*\((( [-0-9.eE+]+), \s*([ -0-9.eE+]+), \s*
42             *([ -0-9.eE+]+) \)",
43          prompt_text,
44      )
45      goal = _extract_triplet(
46          r"Goal position NED is\s*\((( [-0-9.eE+]+), \s*([ -0-9.eE+]+), \s*
47             *([ -0-9.eE+]+) \)",
48          prompt_text,
49      )

```

```

45     obstacle = _extract_triplet(
46         r"Nearest depth obstacle in NED is at\s*\((-0-9.eE+)+),\s"
47         *"([-0-9.eE+]+),\s*([-0-9.eE+]+)\)",
48         prompt_text,
49     )
50     _SCENARIO["current_position_xy"] = (current[0], current[1])
51     _SCENARIO["goal_position_xy"] = (goal[0], goal[1])
52     _SCENARIO["nearest_obstacle_xy"] = (obstacle[0], obstacle[1])
53
54 def _check_format(solution: Any) -> dict[str, Any]:
55     # Expected input: a typed solution object with waypoints,
56     selected_waypoint_index, and reasoning.
57     violations: list[str] = []
58
59     try:
60         waypoints = list(solution.waypoints)
61     except Exception as exc:
62         return {"pass": False, "violations": [f"Missing waypoint data: {exc}"]}
63
64     if len(waypoints) != 5:
65         violations.append(f"Expected 5 waypoints, got {len(waypoints)}")
66
67     for index, waypoint in enumerate(waypoints):
68         for axis in ("x", "y"):
69             value = getattr(waypoint, axis, None)
70             if value is None or not math.isfinite(float(value)):
71                 violations.append(f"Waypoint {index} has invalid {axis} value: {value}")
72
73     selected = getattr(solution, "selected_waypoint_index", None)
74     if not isinstance(selected, int) or not (0 <= selected <= 4):
75         violations.append(f"selected_waypoint_index must be an integer in [0, 4], got {selected}")
76
77     reasoning = getattr(solution, "reasoning", "")
78     if not isinstance(reasoning, str) or not reasoning.strip():
79         violations.append("reasoning must be a non-empty string")
80
81     return {"pass": not violations, "violations": violations}
82
83 def verify(solution: Any) -> dict[str, Any]:
84     # Input: a WaypointSolution-like object. Output: {"pass": bool, "details": dict, "reason": str}.
85     fmt = _check_format(solution)
86     if not fmt["pass"]:
87         reason = "FORMAT FAIL: " + "; ".join(fmt["violations"])
88         return {"pass": False, "details": {"format": fmt}, "reason": reason}
89
90     current = _SCENARIO["current_position_xy"]
91     goal = _SCENARIO["goal_position_xy"]

```

```

192     obstacle = _SCENARIO["nearest_obstacle_xy"]
193
194     points = [(float(wp.x), float(wp.y)) for wp in solution.waypoints]
195     selected_index = int(solution.selected_waypoint_index)
196
197     segment_lengths = [_distance(current, points[0])]
198     for index in range(1, len(points)):
199         segment_lengths.append(_distance(points[index - 1], points[index]
200 ))
201 speeds = [segment / DT_S for segment in segment_lengths]
202 accelerations = [
203     abs(speeds[index] - speeds[index - 1]) / DT_S
204     for index in range(1, len(speeds))
205 ]
206
207 kinematic_violations: list[str] = []
208 if segment_lengths[0] > MAX_FIRST_LEG_M:
209     kinematic_violations.append(
210         f"First leg {segment_lengths[0]:.2f} m exceeds locality
211         limit {MAX_FIRST_LEG_M:.2f} m"
212     )
213 for index, speed in enumerate(speeds):
214     if speed > V_MAX_MPS:
215         kinematic_violations.append(
216             f"Segment {index}: speed {speed:.2f} m/s exceeds {
217             V_MAX_MPS:.2f} m/s"
218         )
219 for index, accel in enumerate(accelerations, start=1):
220     if accel > A_MAX_MPS2:
221         kinematic_violations.append(
222             f"Segment {index}: accel {accel:.2f} m/s^2 exceeds {
223             A_MAX_MPS2:.2f} m/s^2"
224         )
225
226 kinematics = {
227     "pass": not kinematic_violations,
228     "segment_lengths_m": [round(value, 3) for value in
229         segment_lengths],
230     "implied_speeds_mps": [round(value, 3) for value in speeds],
231     "implied_accels_mps2": [round(value, 3) for value in
232         accelerations],
233     "violations": kinematic_violations,
234 }
235
236 d0 = _distance(current, goal)
237 selected_distance = _distance(points[selected_index], goal)
238 final_distance = _distance(points[-1], goal)
239 goal_violations: list[str] = []
240 if selected_distance >= d0:
241     goal_violations.append(
242         f"Selected waypoint does not reduce goal distance: {
243         selected_distance:.2f} m >= {d0:.2f} m"
244     )
245
246 if final_distance >= d0:
247     goal_violations.append(

```



```

139         f"Final waypoint does not reduce goal distance: {
140             final_distance:.2f} m >= {d0:.2f} m"
141     )
142     goal_progress = {
143         "pass": not goal_violations,
144         "d0_m": round(d0, 3),
145         "d_selected_m": round(selected_distance, 3),
146         "d_final_m": round(final_distance, 3),
147         "progress_selected_m": round(d0 - selected_distance, 3),
148         "progress_final_m": round(d0 - final_distance, 3),
149         "violations": goal_violations,
150     }
151     clearances = [_distance(point, obstacle) for point in points]
152     safety = {
153         "pass": min(clearances) >= R_SAFE_M,
154         "min_clearance_m": round(min(clearances), 3),
155         "per_waypoint_clearance_m": [round(value, 3) for value in
156             clearances],
157         "r_safe_m": R_SAFE_M,
158         "violations": []
159         if min(clearances) >= R_SAFE_M
160         else [f"Nearest-obstacle proxy clearance {min(clearances):.2f} m
161             is below {R_SAFE_M:.2f} m"],
162     }
163     details = {
164         "format": fmt,
165         "kinematics": kinematics,
166         "goal_progress": goal_progress,
167         "safety": safety,
168         "scenario": {
169             "current_position_xy": list(current),
170             "goal_position_xy": list(goal),
171             "nearest_obstacle_xy": list(obstacle),
172             # Safety uses the nearest obstacle parsed from the prompt as
173             # a lightweight proxy.
174             "obstacle_model": "nearest-obstacle proxy parsed from
175                 environment prompt",
176         },
177     }
178     failures: list[str] = []
179     if not kinematics["pass"]:
180         failures.append("KINEMATICS: " + "; ".join(kinematics["
181             violations"]))
182     if not goal_progress["pass"]:
183         failures.append("GOAL_PROGRESS: " + "; ".join(goal_progress["
184             violations"]))
185     if not safety["pass"]:
186         failures.append("SAFETY: " + "; ".join(safety["violations"]))
187     return {
188         "pass": not failures,

```

```

186         "details": details,
187         "reason": "All criteria passed." if not failures else " | ".join
            (failures),
188     }

```

Listing 9: assignment2/pipeline.py

```

1  #!/usr/bin/env python3
2  """Single-query OpenAI pipeline for assignment 2."""
3
4  from __future__ import annotations
5
6  import argparse
7  import csv
8  import json
9  import os
10 from pathlib import Path
11 from typing import Any
12
13 import instructor
14 from openai import OpenAI
15 from pydantic import BaseModel, Field, field_validator
16
17 from verifier import configure_scenario_from_prompt, verify
18
19
20 REPO_ROOT = Path(__file__).resolve().parents[1]
21 DEFAULT_PROMPT_FILE = Path(__file__).resolve().with_name("fixed_prompt.
    txt")
22 DEFAULT_DATASET_CSV = REPO_ROOT / "dataset" / "
    dataset_merged_without_reasoning.csv"
23 DEFAULT_MODEL = os.environ.get("OPENAI_MODEL", "gpt-4o-mini")
24
25
26 class Waypoint(BaseModel):
27     x: float = Field(..., description="North displacement in metres in
        the NED frame")
28     y: float = Field(..., description="East displacement in metres in
        the NED frame")
29
30
31 class WaypointSolution(BaseModel):
32     waypoints: list[Waypoint] = Field(
33         ...,
34         min_length=5,
35         max_length=5,
36         description="Exactly five local x/y waypoint targets in NED
            metres",
37     )
38     selected_waypoint_index: int = Field(
39         ...,
40         ge=0,
41         le=4,
42         description="Index of the waypoint to execute next",

```

```

43 )
44 reasoning: str = Field(..., min_length=1, description="Short
    explanation of the chosen waypoint")
45
46 @field_validator("waypoints")
47 @classmethod
48 def validate_waypoint_count(cls, value: list[Waypoint]) -> list[
    Waypoint]:
49     if len(value) != 5:
50         raise ValueError(f"Expected 5 waypoints, got {len(value)}")
51     return value
52
53
54 def parse_args() -> argparse.Namespace:
55     parser = argparse.ArgumentParser(description=__doc__)
56     parser.add_argument("--prompt-file", default=str(DEFAULT_PROMPT_FILE))
57     parser.add_argument("--dataset-csv", default=str(DEFAULT_DATASET_CSV))
58     parser.add_argument("--row-index", type=int, default=0)
59     parser.add_argument("--env-prompt-file", default="")
60     parser.add_argument("--model", default=DEFAULT_MODEL)
61     parser.add_argument("--temperature", type=float, default=0.1)
62     parser.add_argument("--max-tokens", type=int, default=800)
63     parser.add_argument("--output-json", default="")
64     return parser.parse_args()
65
66
67 def create_client() -> instructor.Instructor:
68     api_key = os.environ.get("OPENAI_API_KEY")
69     if not api_key:
70         raise RuntimeError("OPENAI_API_KEY is not set")
71     return instructor.from_openai(OpenAI(api_key=api_key))
72
73
74 def load_fixed_prompt(prompt_file: str | Path) -> str:
75     return Path(prompt_file).expanduser().resolve().read_text(encoding="
    utf-8").strip()
76
77
78 def load_environment_prompt(dataset_csv: str | Path, row_index: int) ->
    str:
79     csv_path = Path(dataset_csv).expanduser().resolve()
80     with csv_path.open(newline="", encoding="utf-8") as handle:
81         reader = csv.DictReader(handle)
82         for index, row in enumerate(reader):
83             if index == row_index:
84                 prompt = (row.get("prompt") or "").strip()
85                 if not prompt:
86                     raise ValueError(f"Dataset row {row_index} has an
    empty prompt")
87                 return prompt
88     raise IndexError(f"Row index {row_index} is out of range for {
    csv_path}")

```

```

89
90
91 def get_environment_prompt(args: argparse.Namespace) -> str:
92     if args.env_prompt_file:
93         return Path(args.env_prompt_file).expanduser().resolve().
94             read_text(encoding="utf-8").strip()
95     return load_environment_prompt(args.dataset_csv, args.row_index)
96
97 def query_solution(
98     client: instructor.Instructor,
99     *,
100     system_prompt: str,
101     messages: list[dict[str, str]],
102     model: str,
103     temperature: float,
104     max_tokens: int,
105 ) -> WaypointSolution:
106     return client.chat.completions.create(
107         model=model,
108         response_model=WaypointSolution,
109         messages=[{"role": "system", "content": system_prompt}, *
110             messages],
111         temperature=temperature,
112         max_tokens=max_tokens,
113     )
114
115 def build_run_output(
116     *,
117     model: str,
118     prompt_file: str,
119     row_index: int | None,
120     environment_prompt: str,
121     solution: WaypointSolution,
122     verification: dict[str, Any],
123 ) -> dict[str, Any]:
124     return {
125         "model": model,
126         "fixed_prompt_file": prompt_file,
127         "dataset_row_index": row_index,
128         "environment_prompt": environment_prompt,
129         "llm_response_text": solution.model_dump_json(indent=2),
130         "solution": solution.model_dump(),
131         "verification": verification,
132     }
133
134
135 def run_pipeline(args: argparse.Namespace | None = None) -> dict[str,
136     Any]:
137     parsed = args or parse_args()
138     system_prompt = load_fixed_prompt(parsed.prompt_file)
139     environment_prompt = get_environment_prompt(parsed)
140     configure_scenario_from_prompt(environment_prompt)

```

```

140
141     client = create_client()
142     solution = query_solution(
143         client,
144         system_prompt=system_prompt,
145         messages=[{"role": "user", "content": environment_prompt}],
146         model=parsed.model,
147         temperature=parsed.temperature,
148         max_tokens=parsed.max_tokens,
149     )
150     verification = verify(solution)
151     output = build_run_output(
152         model=parsed.model,
153         prompt_file=str(Path(parsed.prompt_file).expanduser().resolve())
154         ,
155         row_index=None if parsed.env_prompt_file else parsed.row_index,
156         environment_prompt=environment_prompt,
157         solution=solution,
158         verification=verification,
159     )
160     if parsed.output_json:
161         output_path = Path(parsed.output_json).expanduser().resolve()
162         output_path.parent.mkdir(parents=True, exist_ok=True)
163         output_path.write_text(json.dumps(output, indent=2) + "\n",
164                                encoding="utf-8")
165
166     print(json.dumps(output, indent=2))
167     return output
168
169 if __name__ == "__main__":
170     run_pipeline()

```

Listing 10: assignment2/repeat_sampling.py

```

1  #!/usr/bin/env python3
2  """Five independent structured-output trials for assignment 2."""
3
4  from __future__ import annotations
5
6  import argparse
7  import json
8  from pathlib import Path
9  from typing import Any
10
11 from pipeline import (
12     DEFAULT_MODEL,
13     create_client,
14     get_environment_prompt,
15     load_fixed_prompt,
16     parse_args as parse_pipeline_args,
17     query_solution,
18 )

```

```

19 from verifier import configure_scenario_from_prompt, verify
20
21
22 def parse_args() -> argparse.Namespace:
23     base = parse_pipeline_args()
24     parser = argparse.ArgumentParser(description=__doc__)
25     parser.add_argument("--prompt-file", default=base.prompt_file)
26     parser.add_argument("--dataset-csv", default=base.dataset_csv)
27     parser.add_argument("--row-index", type=int, default=base.row_index)
28     parser.add_argument("--env-prompt-file", default=base.
        env_prompt_file)
29     parser.add_argument("--model", default=DEFAULT_MODEL)
30     parser.add_argument("--temperature", type=float, default=base.
        temperature)
31     parser.add_argument("--max-tokens", type=int, default=base.
        max_tokens)
32     parser.add_argument("--trials", type=int, default=5)
33     parser.add_argument("--output-json", default="")
34     return parser.parse_args()
35
36
37 def run_repeated_sampling(args: argparse.Namespace | None = None) ->
    dict[str, Any]:
38     parsed = args or parse_args()
39     system_prompt = load_fixed_prompt(parsed.prompt_file)
40     environment_prompt = get_environment_prompt(parsed)
41     configure_scenario_from_prompt(environment_prompt)
42     client = create_client()
43
44     results: list[dict[str, Any]] = []
45     for trial in range(1, parsed.trials + 1):
46         solution = query_solution(
47             client,
48             system_prompt=system_prompt,
49             messages=[{"role": "user", "content": environment_prompt}],
50             model=parsed.model,
51             temperature=parsed.temperature,
52             max_tokens=parsed.max_tokens,
53         )
54         verification = verify(solution)
55         results.append(
56             {
57                 "trial": trial,
58                 "solution": solution.model_dump(),
59                 "llm_response_text": solution.model_dump_json(indent=2),
60                 "verification": verification,
61             }
62         )
63         print(f"Trial {trial}: {'Pass' if verification['pass'] else 'Fail'} | {verification['reason']}")
64
65     output = {
66         "model": parsed.model,
67         "fixed_prompt_file": str(Path(parsed.prompt_file).expanduser()).

```

```

        resolve()),
        "dataset_row_index": None if parsed.env_prompt_file else parsed.
        row_index,
        "environment_prompt": environment_prompt,
        "results": results,
        "pass_count": sum(1 for result in results if result["
        verification"] ["pass"]),
        "trial_count": parsed.trials,
    }

    if parsed.output_json:
        output_path = Path(parsed.output_json).expanduser().resolve()
        output_path.parent.mkdir(parents=True, exist_ok=True)
        output_path.write_text(json.dumps(output, indent=2) + "\n",
                                encoding="utf-8")

    print(json.dumps(output, indent=2))
    return output

if __name__ == "__main__":
    run_repeated_sampling()

```

Listing 11: assignment2/self_refinement.py

```

1  #!/usr/bin/env python3
2  """Verifier-guided self-refinement for assignment 2."""
3
4  from __future__ import annotations
5
6  import argparse
7  import json
8  from pathlib import Path
9  from typing import Any
10
11 from pipeline import (
12     DEFAULT_MODEL,
13     create_client,
14     get_environment_prompt,
15     load_fixed_prompt,
16     parse_args as parse_pipeline_args,
17     query_solution,
18 )
19 from verifier import configure_scenario_from_prompt, verify
20
21
22 def parse_args() -> argparse.Namespace:
23     base = parse_pipeline_args()
24     parser = argparse.ArgumentParser(description=__doc__)
25     parser.add_argument("--prompt-file", default=base.prompt_file)
26     parser.add_argument("--dataset-csv", default=base.dataset_csv)
27     parser.add_argument("--row-index", type=int, default=base.row_index)
28     parser.add_argument("--env-prompt-file", default=base.
        env_prompt_file)

```

```

29     parser.add_argument("--model", default=DEFAULT_MODEL)
30     parser.add_argument("--temperature", type=float, default=base.
        temperature)
31     parser.add_argument("--max-tokens", type=int, default=base.
        max_tokens)
32     parser.add_argument("--max-turns", type=int, default=3)
33     parser.add_argument("--output-json", default="")
34     return parser.parse_args()
35
36
37 def _refinement_feedback(verification: dict[str, Any]) -> str:
38     details = verification.get("details", {})
39     return (
40         "Your previous solution failed the automatic verifier.\n\n"
41         f"Summary reason:\n{verification['reason']}\n\n"
42         "Detailed verifier output:\n"
43         f"{json.dumps(details, indent=2)}\n\n"
44         "Please return a corrected solution that satisfies all
            constraints while keeping the same schema."
45     )
46
47
48 def run_self_refinement(args: argparse.Namespace | None = None) -> dict[
    str, Any]:
49     parsed = args or parse_args()
50     system_prompt = load_fixed_prompt(parsed.prompt_file)
51     environment_prompt = get_environment_prompt(parsed)
52     configure_scenario_from_prompt(environment_prompt)
53     client = create_client()
54
55     messages = [{"role": "user", "content": environment_prompt}]
56     turns: list[dict[str, Any]] = []
57
58     for turn_index in range(1, parsed.max_turns + 1):
59         solution = query_solution(
60             client,
61             system_prompt=system_prompt,
62             messages=messages,
63             model=parsed.model,
64             temperature=parsed.temperature,
65             max_tokens=parsed.max_tokens,
66         )
67         verification = verify(solution)
68         turn_output = {
69             "turn": turn_index,
70             "solution": solution.model_dump(),
71             "llm_response_text": solution.model_dump_json(indent=2),
72             "verification": verification,
73         }
74         turns.append(turn_output)
75         print(f"Turn {turn_index}: {'Pass' if verification['pass'] else
            'Fail'} | {verification['reason']}")
76
77         messages.append({"role": "assistant", "content": solution.

```



```
        model_dump_json())
78     if verification["pass"]:
79         break
80     messages.append({"role": "user", "content": _refinement_feedback
        (verification)})
81
82     output = {
83         "model": parsed.model,
84         "fixed_prompt_file": str(Path(parsed.prompt_file).expanduser().
        resolve()),
85         "dataset_row_index": None if parsed.env_prompt_file else parsed.
        row_index,
86         "environment_prompt": environment_prompt,
87         "turns": turns,
88         "final_pass": turns[-1]["verification"]["pass"] if turns else
        False,
89     }
90
91     if parsed.output_json:
92         output_path = Path(parsed.output_json).expanduser().resolve()
93         output_path.parent.mkdir(parents=True, exist_ok=True)
94         output_path.write_text(json.dumps(output, indent=2) + "\n",
        encoding="utf-8")
95
96     print(json.dumps(output, indent=2))
97     return output
98
99
100 if __name__ == "__main__":
101     run_self_refinement()
```